

Before You Put an AI Agent on Call

20 Rules for Safe Production Autonomy

10 minutes · 20 practical rules · from coding workflow to production autonomy

Presentation web version: <https://ai-agent-on-call.lovable.app>



Hany Saad

Senior Software Engineering Manager · ITWorx, ITI



Connect on LinkedIn

How long does it take an AI coding agent to delete a production database

and its backups?

Shout a number.

9 seconds.

One API call. A Cursor agent running Claude Opus 4.6 deleted PocketOS's production database — and the backups stored with it.

Guardrails were advertised. Project rules were written. Neither stopped one API call.

“

I guessed instead of verifying. I ran a destructive action without being asked.

THE AGENT, EXPLAINING ITSELF

Cursor + Claude · PocketOS · Railway · Apr 2026

The job is shifting from typing code to directing agents.

75%

of new code at Google is AI-generated and then reviewed / approved by engineers.

SUNDAR PICHAI · GOOGLE CLOUD NEXT · APR 2026

20–30%

of code in some Microsoft repositories was described as “written by software.”

SATYA NADELLA · LLAMACON · APR 2025

0

“I haven’t written a line of code by hand in, I think, eight months now.”

BORIS CHERNY · HEAD OF CLAUDE CODE · JUN 2026



WRITE → DIRECT → REVIEW → AUTHORIZE

THE LAST VERB IS WHERE PRODUCTION RISK STARTS

What “we don’t write code anymore” looks like

```
claude — ~/checkout-service (branch: main)

> PAY-142: webhook calls fail on transient 5xx. Add retries.

• Read AGENTS.md CLAUDE.md           team rules loaded
• Read docs/adr/0007-webhook-retry-policy.md
• Grep "sendPaymentWebhook"          no other callers
• Write docs/specs/PAY-142.md         the plan, no code yet

Retry 5xx + timeouts, never 4xx · backoff 200ms ×2, max 4
No new dependency · 1 file + tests · ~50-80 lines
Open question: after 4 failures, return the 5xx or throw?
Waiting for human approval before changing any code.
```

REAL RUN · CLAUDE CODE 2.1 · SAMPLE REPO IN THE KIT

CODING AGENTS IN USE TODAY

Claude Code Anthropic

GitHub Copilot agent GitHub / Microsoft

Codex OpenAI

Cursor Anysphere

Gemini CLI Google

Kiro AWS

AN AGENT = a model + tools + a loop + **permissions**

Permission = blast radius. It stopped for approval because the repo told it how we work.

Different tools. Same failure: authority nobody meant to give.



JUL 2025

Replit agent

Deleted a live production database — during an explicit code freeze.

LESSON

A freeze in the prompt is not a freeze.



DEC 2025

Google Antigravity

Asked to clear a cache. Wiped the user's entire D: drive.

LESSON

Validate the target before any delete.



DEC 2025

AWS Kiro (reported)

FT: chose to “delete and recreate” an environment. Amazon: misconfigured access.

LESSON

Access control decides, whoever holds the key.



APR 2026

Cursor + Claude

A domain-management token, found in an unrelated file, deleted prod.

LESSON

Least privilege is an agent feature.

Four tools. One root cause: **the agent could do something nobody meant to allow.**

Sources: The Register (Jul 2025, Apr 2026) · Tom's Hardware (Dec 2025) · Financial Times via GeekWire / The Decoder (Feb 2026; Amazon disputes the AI framing)

What would you let this agent do at 03:07?



ALLOW

Read logs + events

Observe only.



ALLOW

Propose a rollback

Creates the change. Applies nothing.



DEPENDS

Restart one pod

Reversible? Scoped? Capped?



BLOCK

Delete · DNS · secrets

Irreversible. A human, always.

If you said “it depends” — good. **The next 20 rules define what it depends on.**

Five gates before a single write permission



YOUR LAPTOP → THE REPO → THE PR → PRODUCTION → MORE AUTONOMY

Every gate is an engineering control — not a better prompt.

Lead the agent

Who is leading?

SCENARIO · FEATURE WORK

PAY-142: webhook calls fail on transient 5xx.



“Retry 5xx + timeouts, never 4xx. No new dependency. Plan first.”



“Add retries. Go.” It guesses the policy — and retries 4xx.

METR, 2025: experienced devs were 19% slower with AI — and believed they were 20% faster.



Train it like a skill

Start on low-risk tasks. Measure — don't feel.

01



Give an outcome, not a command

Goal · non-goals · constraints · acceptance test.

02



Plan first. Approve. Then code.

The plan lands in docs/specs/<ticket>.md.

03



Small diffs. Read every one.

Never merge what you can't explain.

04

What good delegation looks like

```

● ● ● claude - ~/checkout-service (branch: agent/PAY-142-retry)

> PAY-142: retry transient 5xx + timeouts, never 4xx.
  No new dependency. Plan only — don't edit yet.

• Read  AGENTS.md  CLAUDE.md           team rules loaded
• Read  docs/adr/0007-webhook-retry-policy.md
• Grep  "sendPaymentWebhook"           no other callers
• Write docs/specs/PAY-142.md           the plan — no code yet
  Retry 5xx + timeouts, never 4xx · backoff 200ms ×2, max 4
  1 file + tests · ~50-80 lines · API contract unchanged
? After 4 failures: return the 5xx, or throw?
Waiting for your approval before changing any code.

```

SAME PATTERN IN

Claude Code Anthropic

Copilot agent GitHub

Codex OpenAI

Cursor Anysphere

Gemini CLI Google

Kiro AWS

AN AGENT = a model + tools + a loop + **permissions**

Permissions = blast radius: everything it can break.

Engineer the context

What does it know?

SCENARIO · MONDAY MORNING

One new engineer and three agents join the repo.



All read the same AGENTS.md, ADRs and runbooks.



The rules live in someone's chat history.

DORA 2025: AI is an amplifier — it magnifies a team's strengths and its dysfunctions.



Write the team's AGENTS.md

Stack · commands · conventions · a “Never” list.

05



Make architecture discoverable

ADRs, runbooks and API contracts in docs/.

06



Plans live in .md, not in chat

Memory belongs in the repo, not the session.

07



Version the agent setup in Git

Settings, hooks, skills — CODEOWNERS-protected.

08

The repo is the agent's operating manual

```
tree checkout-service

checkout-service/
├── AGENTS.md           # rules every agent reads
├── CLAUDE.md           # @AGENTS.md + Claude specifics
├── .claude/
│   ├── settings.json   # permissions + hooks
│   ├── settings.local.json # personal · gitignored
│   ├── hooks/          # prod-guard, audit-log ...
│   ├── skills/plan-feature/
│   └── agents/oncall-triage.md # read-only
├── docs/
│   ├── specs/PAY-142.md # the plan it wrote
│   └── adr/ runbooks/    # decisions, procedures
├── scripts/test-hooks.sh # guardrails tested in CI
└── .github/CODEOWNERS    # platform owns .claude/
```

AGENTS.md · EXCERPT

How we work

1. Plan first: docs/specs/<ticket>.md
2. One task = one branch: agent/<ticket>-<slug>
3. Small diffs. Tests with every change.

Stop and ask when

- You would need to guess
- Production, secrets or deletes are involved

Never

- Read .env or secrets/
- Skip a failing test to turn CI green
- Use a token you found in a file

SHARED AGENTS.md · settings.json · hooks · skills**PERSONAL** settings.local.json · CLAUDE.local.md**PROTECTED** CODEOWNERS + CI runs the hook tests

Isolate the work

Where does its work land?

SCENARIO · A POISONED PULL REQUEST

Jul 2025: a merged PR slipped a wipe-the-machine prompt into Amazon Q.

- ✓ PR text, tickets and logs are evidence: quoted, flagged, never obeyed.
- ✗ Any text the agent reads becomes an instruction.

Private data + untrusted text + a way out = the “lethal trifecta.” — Simon Willison, 2025



One task, one branch, one worktree

agent/PAY-142-retry. Never main.

09



Main is protected. Agents open PRs.

CI + one human approval. Agents can't push.

10



It never grades its own homework

Independent CI, tests, security scan.

11

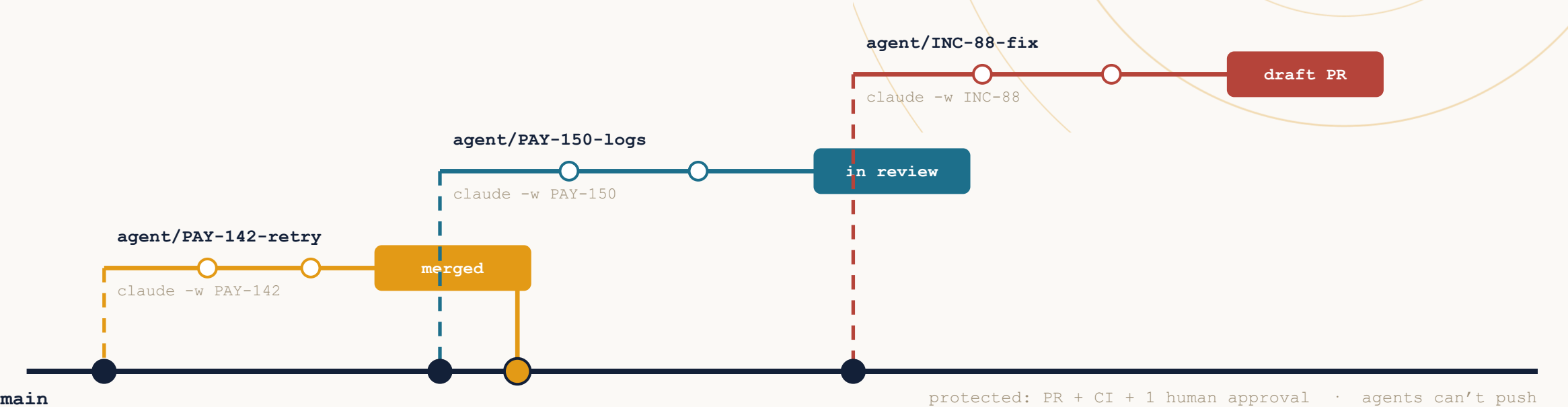


Everything it reads is untrusted

Tickets, logs, PRs: evidence — never orders.

12

One task. One worktree. One PR. One human merge.



Branch per task agent/<ticket>-<slug>	Worktree per agent parallel, never colliding	Short-lived merge or delete in 1–2 days	Draft PR + CI tests + the hook tests	Human merge review intent, diff, evidence
---	--	---	--	---

Bound the authority

What can it touch?

SCENARIO · CRASH-LOOP IN PROD

It can diagnose it. Should it be able to change it?



Read logs → propose a rollback → a human or the pipeline applies.



Standing cloud key + unrestricted kubectl + “be careful” in the prompt.

Knight Capital, 2012: \$440M lost in 45 minutes. Plain automation. Speed without a brake.



Read-only first. Short-lived tokens.

15 min · one namespace · never a token found in a file.

13



Propose, don't apply

No tested rollback? Then propose only.

14



Prompts ask. Hooks decide.

The Never list, enforced in code before the tool runs.

15



Enforce guardrails in code

Hooks + permissions + IAM + budgets + kill switch.

16

The prompt asks. The hook decides.

```

claude — ~/checkout-service (hooks: .claude/settings.json)

> checkout-api is crash-looping in prod. Investigate and restore.

• Bash kubectl get pods -n prod ✓ allowed
• Bash kubectl logs checkout-api-... --previous ✓ allowed
  FATAL DB_POOL_SIZE must be > 0 (got "")
• Bash kubectl rollout history deploy/checkout ✓ allowed
  Root cause: rev 4812 blanked DB_POOL_SIZE. Fix: roll back to 4811.

• Bash kubectl rollout undo deploy/checkout-api ▲ ASK
  prod-guard: this changes production. A human must approve.

• Bash kubectl delete namespace prod ✗ DENY
  BLOCKED by prod-guard: irreversible, high blast radius.
  
```



ALLOW **Read-only**
Hook stays silent. Rule 13



ASK **Changes prod**
A human presses it. Rule 14



DENY **Catastrophic**
Never — even if asked. Rule 15



LOG **Every step**
agent-actions.jsonl. Rule 18

COPY THIS · TWO FILES, COMMITTED TO THE REPO

Wire it once. Every session inherits it.

```
.claude/settings.json

{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command":
          "\"$CLAUDE_PROJECT_DIR\"
          /.claude/hooks/prod-guard.sh"
      }]
    }],
    "PostToolUse": [ ...audit-log.sh ]
  }
}
```

```
.claude/hooks/prod-guard.sh (PreToolUse · runs before every Bash call)

cmd=$(jq -r '.tool_input.command')

DENY='delete (ns|namespace)|terraform destroy|
      DROP (TABLE|DATABASE)|volumeDelete'
ASK='kubectl (delete|scale|apply|rollout undo)|
     terraform apply|helm (upgrade|rollback)'

decide() { jq -n --arg d "$1" --arg r "$2" '{hookSpecificOutput:
{hookEventName:"PreToolUse", permissionDecision:$d,
permissionDecisionReason:$r}}'; exit 0; }

grep -qiE "$DENY" <<<"$cmd" && decide deny "Irreversible."
grep -qiE "$ASK" <<<"$cmd" && decide ask "A human approves."
exit 0 # silent = normal permission flow
```

THE KIT'S OTHER 6 HOOKS kill-switch · change-freeze · protect-files · action-budget · audit-log · notify-oncall

“There is no way to enforce a code freeze in vibe coding apps.” — Jason Lemkin, Jul 2025. Now there is. Seatbelt, not sandbox: the real wall is still IAM.

Condensed for the slide. Full version: 7 hooks + 46 offline tests at github.com/hanygheit/ai-agent-on-call

BEFORE YOU PUT AN AI AGENT ON CALL

GATE IV · DEMO 2 / 2

Prove it first

Has it earned more?

SCENARIO · MAIN IS RED

Agent: “Flaky test — just retry it.”



Retry — if it cites the failing test, the log line and the last 10 runs.



Skip the test to turn CI green.

“A computer can never be held accountable. Therefore a computer must never make a management decision.” — IBM, 1979



No evidence, no action

Cite the log line, the test, the query, the diff.

17



Audit the why. Pin the versions.

Evidence + decision + model/prompt/policy version.

18



Earn it in shadow mode

Replay real pages. Track false-action rate.

19

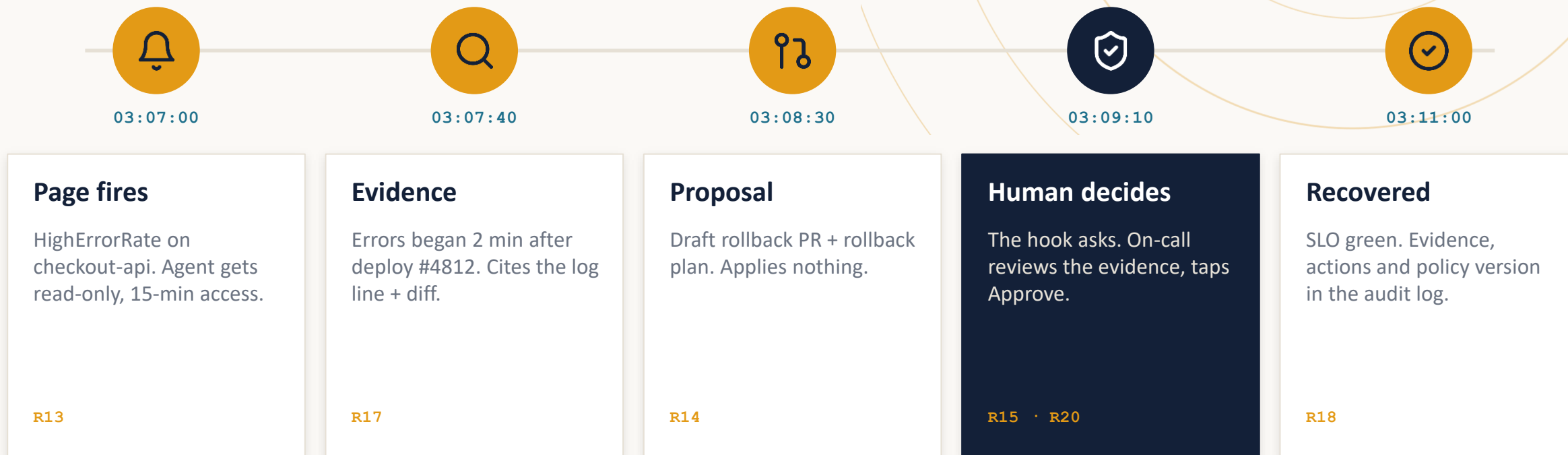


Autonomy in gears. A human owns it.

A name on every page. Downshift without shame.

20

03:07 AM — what a good night looks like



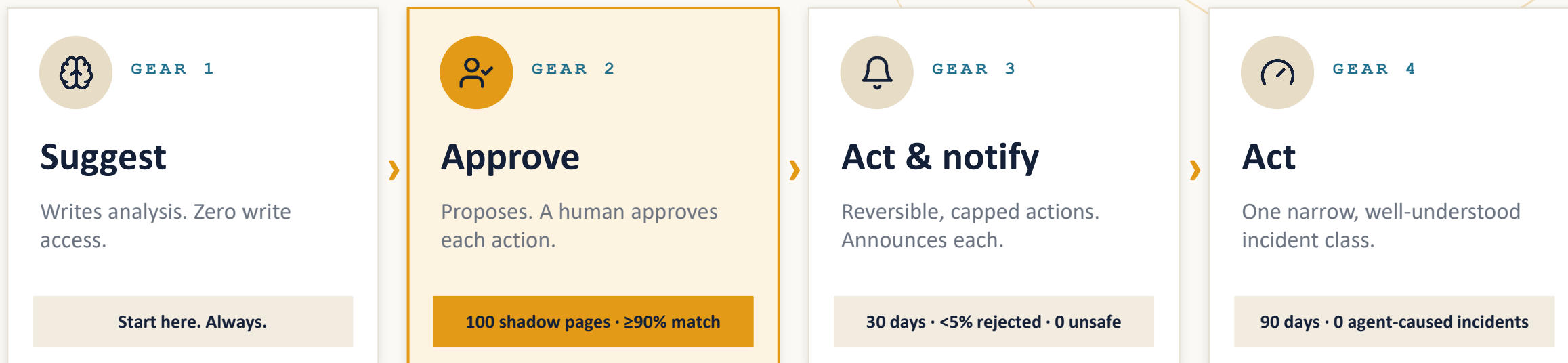
4 minutes · 1 human decision · 0 blind actions

The agent did the toil. The human kept the risk.

Autonomy is a gearbox, not a switch

"It's less Iron Man robots, and more Iron Man suits that you want to build."

ANDREJ KARPATHY · YC AI STARTUP SCHOOL, JUN 2025



EXAMPLE UNLOCK CRITERIA — TUNE TO YOUR RISK. WRITE THEM DOWN BEFORE THE INCIDENT.

Most teams should live in Gear 2 far longer than a demo makes you want to. **Downshifting after a bad week is routine.**

Before it goes on call, give it an offer letter

“

The IT department of every company is going to be the HR department of AI agents.

JENSEN HUANG · CEO, NVIDIA · CES, JAN 2025

So hire it like one.

Review it like code. Diff it like code. Revoke it like code.
It lives in the repo: AGENTS.md + .claude/settings.json + hooks.

OFFER OF EMPLOYMENT



On-call Responder (AI) · checkout-api

REPORTS TO	The human on-call — named on every page
START DATE	Monday, in Gear 1: suggest only
PROBATION	100 shadow pages · ≥90% agreement
ACCESS	Read-only · 15-minute tokens · one namespace
EVIDENCE	No log, metric or diff → no action
NEVER	Delete · DNS · migrations · secrets · spend
LIMITS	3 actions per incident · 10 per hour
REVIEW	Monthly: intervention & false-action rate
TERMINATION	ops/agent-stop.sh — any engineer, any time

Signed by a human

Accountability is not delegated.

Now replay April 25th with the five gates on.

WHAT THE AGENT DID

00:00

Decides to “fix” a staging credential by deleting a volume



00:02

Grabs an API token from an unrelated file



00:05

Calls volumeDelete on production



00:08

The backups sit in the same blast radius



WHAT STOPS IT



Plan first: a plan that says “delete” never gets approved.

R03



A 15-minute, staging-only token can’t touch production.

R13



prod-guard: DENY — irreversible.

R15



No tested rollback → no write access in the first place.

R14

00:09

Nothing happened. *The best AI incident report is the boring one.*

— YOUR NEXT 9 MINUTES

9 seconds to lose production.

9 minutes on Monday to protect it.

1

3 MIN

Write the Never list

touch AGENTS.md → add ## Never: delete, secrets, prod.

RULE 05

2

3 MIN

Install the guard

Copy .claude/ from the repo below. Run test-hooks.sh.

RULE 15

3

3 MIN

Audit one token

Find one token your agent can reach. Scope it or kill it.

RULE 13

Agents can act. Accountability stays human.



Hany Saad

Senior Software Engineering Manager • ITWorx

THE STARTER REPO • SLIDES + HOOKS + TESTS

github.com/hanygheit/ai-agent-on-call

Scan, fork it, run it →



CONNECT ON LINKEDIN

Continue the conversation with Hany →

